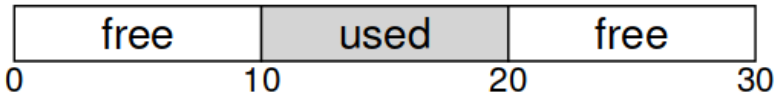
Управление свободным пространством

В этой лекции мы обсудим фундаментальный аспект любой системы управления памятью, не зависимо от того это библиотека malloc (управляющая страницами кучи процесса) или сама ОС (управляющая порциями адресного пространства процесса. В особенности уделим внимание проблемам управления свободным пространством.

Оконкретим проблему. Управление свободным пространством может быть простым в концепции страниц памяти. Довольно легко управлять свободным пространством, когда вся память разбита на кусочки фиксированного размера. В этом случае вы просто храните список этих фиксированных кусочков и когда клиент запрашивает один из них, просто отдаете ему первый свободный.

Более сложное управление памятью начинается тогда, когда свободное пространство состоит из кусочков разной, изменяемой величины. Такие ситуацию возникают при использовании пользовательских библиотек выделеяния памяти (malloc, free) а также при управлении ОС физической памяти, при использовании сегментации для воплощения виртуализации памяти. В каждом из этих случаев возникает проблема известная как **внешняя фрагментация** (external fragmentation): свободное пространство разбивается на маленькие кусочки разных размеров и так получается фрагментация. Последующие запросы на выделение памяти могу провалиться, потому что не будет свободного кусочка памяти требуемого размера, хотя общее количество свободной памяти с головой покрывают запрашиваемый размер.



Изображение демонстрирует описанную проблему. В этом случае свободно 20 байт памяти, но оно разделено на несколько кусочков, по 10 байт каждый. И таким образом запрос на выделение 15 байт может провалиться, хоть и свободно 20 байт.

Суть проблемы:
Как управлять свободным пространством при условии выделения кусочков памяти разного размера. Какие стратегии можно использовать для минимизации фрагментации?

Предположения

Предположим использование стандартной библиотеки malloc и free. При использовании malloc в качестве аргумента передается желаемый размер, а возвращается указатель на выделенную память. Free же в свою очередь в качестве параметра ожидает указатель на освобождаемый регион памяти и он не проинформирован о размере освобождаемого участка. Таким образом библиотека должна определить размер очищаемого пространства, имея на руках только указатель на начало этого участка.

Пространство, которым управляет эта библиотека исторически называется куча, а структура данных, используемая для управления свободным пространством в куче принимает некоторую форму списка (**free list**). Эта структура содержит ссылки на все свободные участки памяти в этом управляемом регионе памяти. Конечно эта структура данных может и не быть именно списком, но должна быть в виде некоторой структуры, отслеживающей свободное пространство.

В дальнейшем мы будем предполагать что нас волнует именно описанная выше внешняя фрагментация. Распределители могут иметь также проблемы **внутренней фрагментации**. Если Распределитель выдает куски памяти больше чем запрашивалось, то та память о которой не просил клиент (и таким образом которая не используется) и есть пример внутренней фрагментации (потому что потери случаются внутри выделенной области памяти) и это является другим примером потерь в памяти.

Также одним из предположений будет то, что единожды выделенное клиенту адресное пространство не может поменять свое расположение в памяти. Например, если программа делает вызов malloc() и получает указатель на выделенный регион памяти, считается что этот регион становится ее собственностью и уже не может быть перемещен до тех пор, пока программа не освободит его с помощью вызова free(). Таким образом невозможно уплотнение свободного пространства, хотя это могло бы помочь в борьбе с фрагментацией.

Наконец, предположим что распределитель управляет только продолжающимся регионом байт. В некоторых случаях распределитель может запросить увеличение этого региона памяти, например программа в случае нехватки памяти может обратиться к ядру с помощью системного вызова sbrk с целью увеличения размера региона. Для упрощения мы будем предполагать что эта возможность не существует.

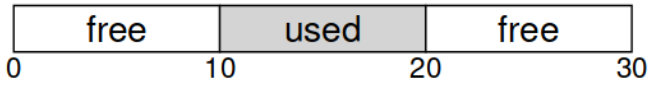
Низкоуровневые механизмы

Перед тем как вникнуть в детали политик, для начала рассмотрим некоторые стандартные механизмы, используемые большинством распределителей. Для начала обсудим основы разделения и объединения, стандартные техники для любых распределителей. Затем рассмотрим как распределители могут быстро отслеживать размеры выделяемых регионов и с относительной простотой. Наконец, обсудим как

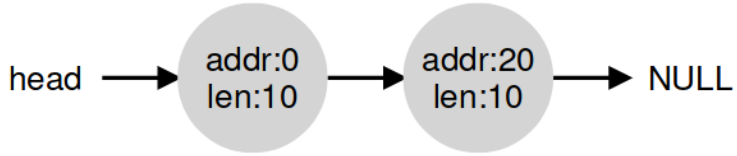
построить простой список внутри свободного пространства для отслеживания того что свободно, а что нет.

Расщепление и Объединение (Splitting and Coalescing)

Free list содержит набор элементов, которые описывают оставшееся свободное пространство в куче. Рассмотрим следующую кучу размером 30 байт:

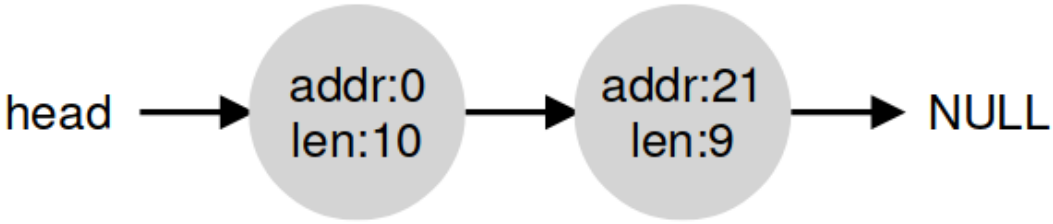


Free list для этой кучи будет состоять из двух элементов. Первая запись будет описывать первый 10-байтовый свободный сегмент (0-9 байт), вторая - второй свободный сегмент (20-29 байт):



Как описано выше, любой запрос более 10 байт будет провальным (вернется NULL). В данном примере просто нет неразрывного куска нужного размера. Запрос размером ровно 10 байт может быть удовлетворен любым из свободных кусков. Но что произойдет, если придет запрос на кусок меньшего размера?

Предположим, что пришел запрос на выделение всего одного байта памяти. В этом случае распределитель выполнит действие, известное как **расщепление**: он найдет свободный кусок памяти, который мог бы удовлетворить запросу и разделит его на два куска. Первый кусок будет отдан запросившему, а второй останется в списке. Таким образом, рассматривая вышеприведенный пример, распределитель мог поступить например следующим образом - он решает выделить запрашиваемый регион памяти во втором свободном куске, вызов malloc() тогда вернет 20 (адрес 1-байтового выделенного региона), а записи во free list приведутся к следующему виду:



На изображении можно заметить, что список практически не изменился, единственное изменение - то что второй свободный регион начинается с 21 вместо 20 и размер этого региона теперь 9 байт. Таким образом расщепления часто используются распределителями, когда приходят запросы на регионы памяти меньшего размера чем любой свободный кусочек памяти.

Как следствие в любом распределителе появился механизм объединяющий свободное пространство. Рассмотрим снова пример в котором первые 10 байт свободны, затем 10 заняты, а затем снова 10 свободны. Что произойдет для заданной кучи если программа сделает вызов free(10), освобождая 10 байт в середине кучи. Если мы просто добавим запись об этом свободном регионе в free list, получится что-то типа этого:



Из такого поведения возникает проблема - теперь даже при условии того что вся куча свободна, она разделена на куски по 10 байт каждая. Таким образом, программа запросившая 20 байт просто не сможет их получить.

Для того чтобы избежать подобных проблем распределители в этом случае объединяют свободные регионы памяти после того как освобождается какой-либо регион памяти. Идея проста: при освобождении программой региона памяти проверить расположенные рядом с ним регионы памяти и если один или несколько таких регионов существует, объединить их в один. И таким образом после объединения наш free list будет выглядеть следующим образом:



В самом деле так и выглядела куча в самом начале, до того как были выделены хоть какие-то регионы памяти. При помощи объединения распределитель может убеждаться что программам можно выделять регионы памяти большего размера.

Отслеживание размера выделенных регионов

Как уже было подмечено ранее, вызов free() принимает в качестве аргумента только указатель, предполагается что библиотека должна быстро определять размер освобождаемого региона и таким образом вернуть свободное пространство во free list.

Для выполнения этой задачи большинство распределителей хранят некоторое количество дополнительной информации в заголовочном блоке, хранимом в памяти, и

обычно помещаемом перед выделяемым регионом памяти.

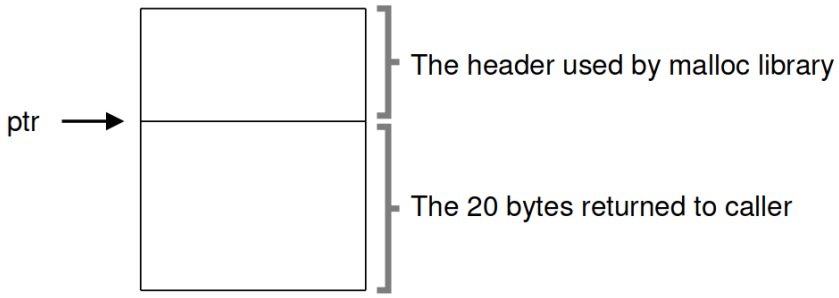


Figure 17.1: **An Allocated Region Plus Header**

Рассмотрим пример. В данной ситуации мы выделяем 20 байт памяти приложению и на этот регион указывает ptr, возвращаемый приложению. Как например если бы пользователь выполнил ptr = malloc(20);

Заголовочный блок хранит как минимум размер выделенного региона (в данном случае 20). Также для увеличения скорости деалокации, он может содержать другие указатели, дополнительную информацию для проверки целостности (magic number), а также другую информацию. Для простоты будем считать что он содержит только magic и size.

```
typedef struct __header_t {
    int size;
    int magic;
} header_t;
```

Таким образом пример выше становится вот таким:

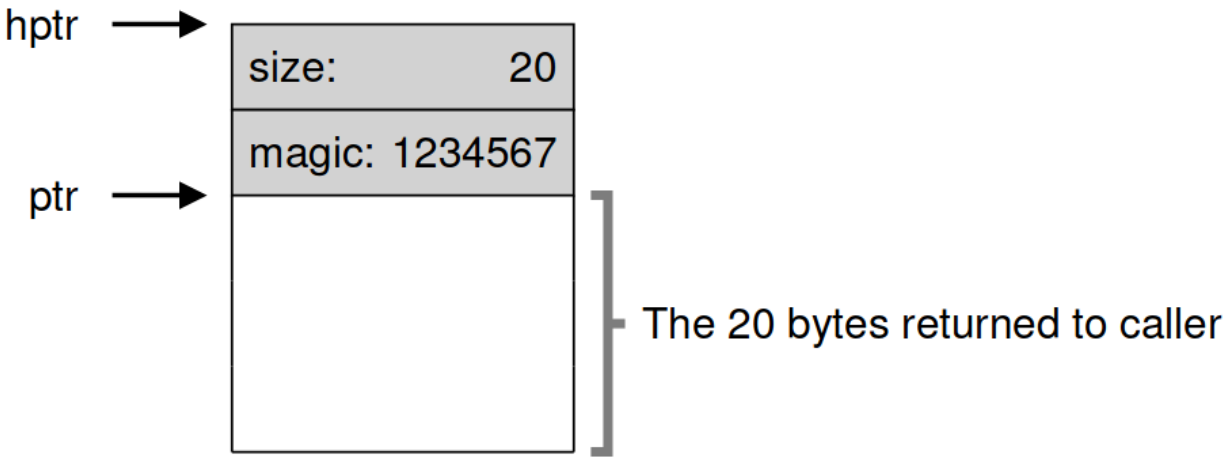


Figure 17.2: **Specific Contents Of The Header**

Когда пользователь вызовет free(ptr), библиотека затем использует указатель и вычисляет адрес начала заголовочного блока:

```
void free(void *ptr) {
    header_t *hptr = (void *)ptr - sizeof(header_t);
    ...
```

После получения такого указателя на заголовочный блок, библиотека может определить и контрольную сумму выделенного региона и с помощью простой математики вычислить размер освобождаемого региона (добавляя размер заголовка к извлеченному из заголовка размеру выделенного региона). Это важная деталь - освобождаемый регион не равен тому числу, который выделял пользователь, он всегда немного больше - размер пользовательского региона плюс размер заголовочного блока.

Embedding A Free List

Как уже говорилось ранее, мы рассматриваем free list как концептуальную сущность. Это просто список, описывающий свободные регионы памяти в куче. Но как справиться с задачей помещения такого списка внутрь самой оперативной памяти?

В типичном списке при выделении новой ноды, вы просто воспользуетесь вызовом malloc() . К сожалению в пределах самой выделяющей библиотеки вы не сможете этого сделать. Вместо этого нужно сформировать free list внутри самого свободного пространства.

Предположим что у нас есть кусочек памяти, с которым мы можем работать, размером 4096 байт. Для того чтобы работать с этой областью памяти в качестве free list мы должны сначала инициализировать этот list. Первоначально у нас должна быть всего одна запись размера 4096-(размер заголовка). Таким образом получаем структуру:

```
typedef struct __node_t {
    int              size;
    struct __node_t *next;
} node_t;
```

Теперь давайте посмотрим на код, который инициализирует кучу и записывает первый элемент списка в это пространство. Мы также предполагаем, что куча построена так, что с обоих сторон у кучи есть еще свободное пространство. Сделать это можно с помощью системного вызова mmap(); Такой способ построения кучи не является единственным, но в данном случае является хорошим примером.

```
// mmap() returns a pointer to a chunk of free space
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size   = 4096 - sizeof(node_t);
head->next   = NULL;
```

После запуска данного кода в нашем free list окажется одна запись, размером 4088 байт. Заголовочный блок содержит адрес, с которого начинается выделенный регион памяти. Представим что этот адрес - 16кб (хотя здесь подойдет любой виртуальный адрес). И в итоге куча будет выглядеть следующим образом:

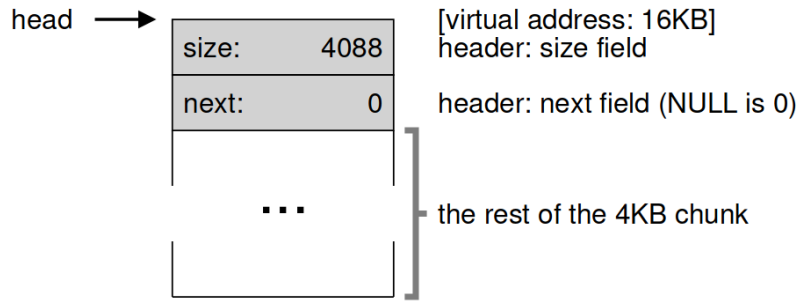


Figure 17.3: **A Heap With One Free Chunk**

Теперь представим что мы выделили некоторый кусочек памяти по запросу программы. Пусть этот выделенный кусочек будет размером 100 байт. Для того чтобы выполнить такой запрос библиотека в первую очередь найдет свободный регион достаточно большого размера. Поскольку в нашем случае такой регион всего один (размером 4088), то он и будет выбран. Затем этот регион будет поделен на 2 куска: один запрашиваемого размера (плюс заголовочный блок), остальное останется свободным. Предполагая, что заголовочный блок имеет размер 8 байт (2 integer числа по 4 байта), распределение пространства в куче будет следующим:

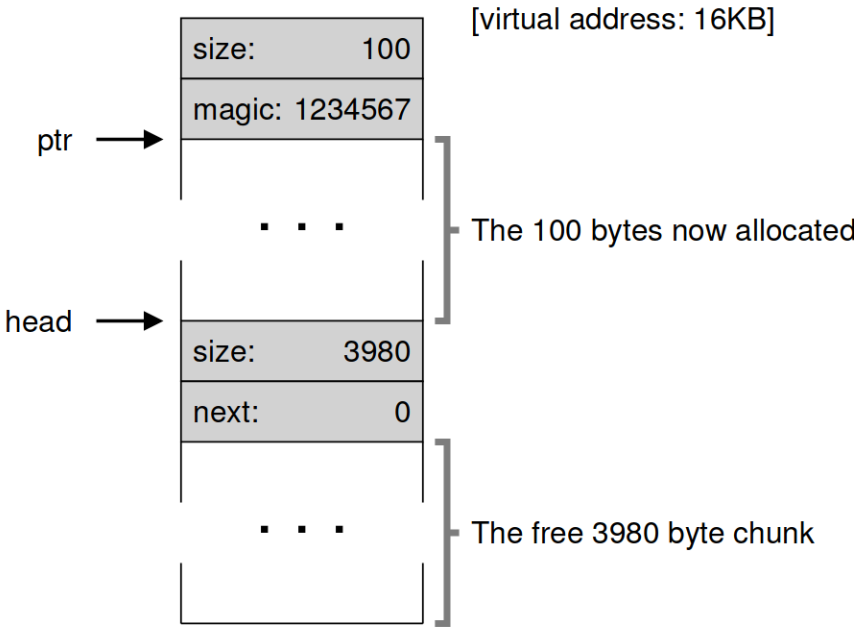


Figure 17.4: **A Heap: After One Allocation**

Таким образом на запрос об выделении 100 байт библиотека выделила 108 байт из свободного региона памяти и вернула указатель на начало выделенного участка) и

переносит указатель head ниже, для последующего использования библиотекой free(), а также записывает новый размер свободного пространства.

Теперь давайте рассмотрим случай, когда выделено сразу 3 региона памяти по 100(+8) байт:

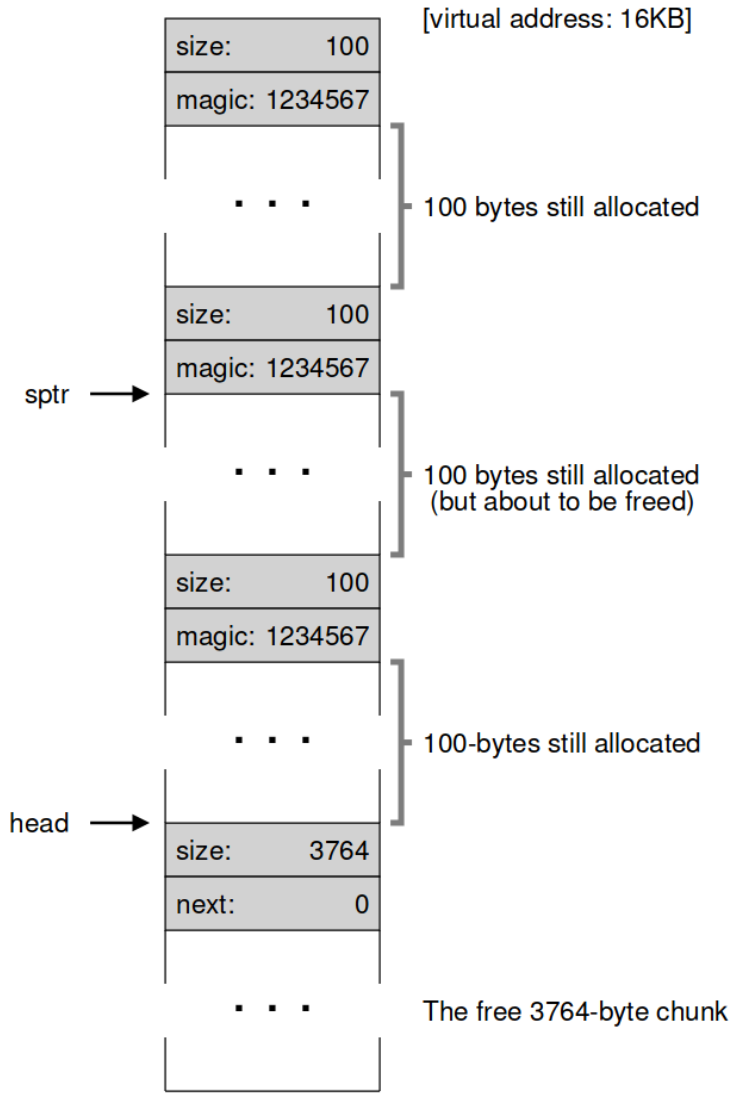


Figure 17.5: **Free Space With Three Chunks Allocated**

В этом случае первые 324 байта заняты, видны заголовочные файлы, но free list остается по-прежнему не истересным - всего одна нода (на которую указывает head), но теперь она имеет размер 3764 байта после 3-х разделений. Но что же произойдет когда какая-либо программа освободит память через вызов free()?

Рассмотрим например, что программа через free(16500) освободила средний кусок памяти. Число 16500 получается следующим образом: 16*1024 + 8 байт заголовка первого региона + 100 байт первого региона + 8 байт заголовка 2го региона. Это и есть тот самый ptr, который получил в качестве возврата malloc().

Библиотека благодаря заголовочному блоку теперь быстро и легко определяет размер освобождаемого региона и добавляет информацию об этом освобожденном куске во free list.

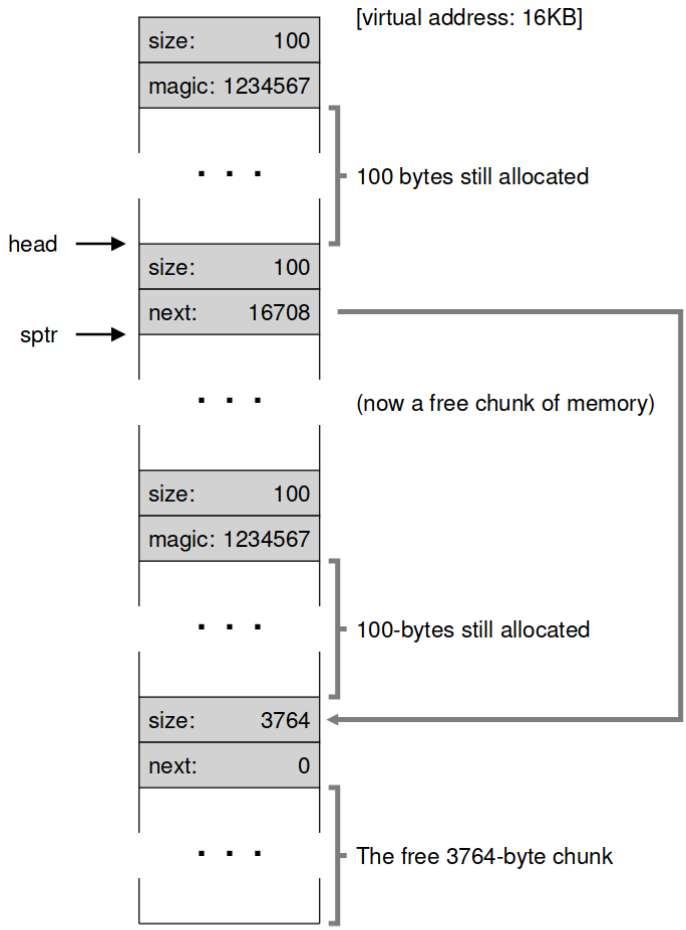


Figure 17.6: **Free Space With Two Chunks Allocated**

Теперь наш free list выглядит следующим образом - он начинается с небольшого свободного куска памяти размером 100байт, на который указывает заголовок этого списка, и большой свободный кусок размером 2764 байта.

Теперь в нашем free list больше одного элемента, а свободное пространство фрагментировано. Сделаем еще одно предположение, что теперь оставшиеся регионы памяти освободились и теперь у нас получается фрагментированный список из 4-х элементов.

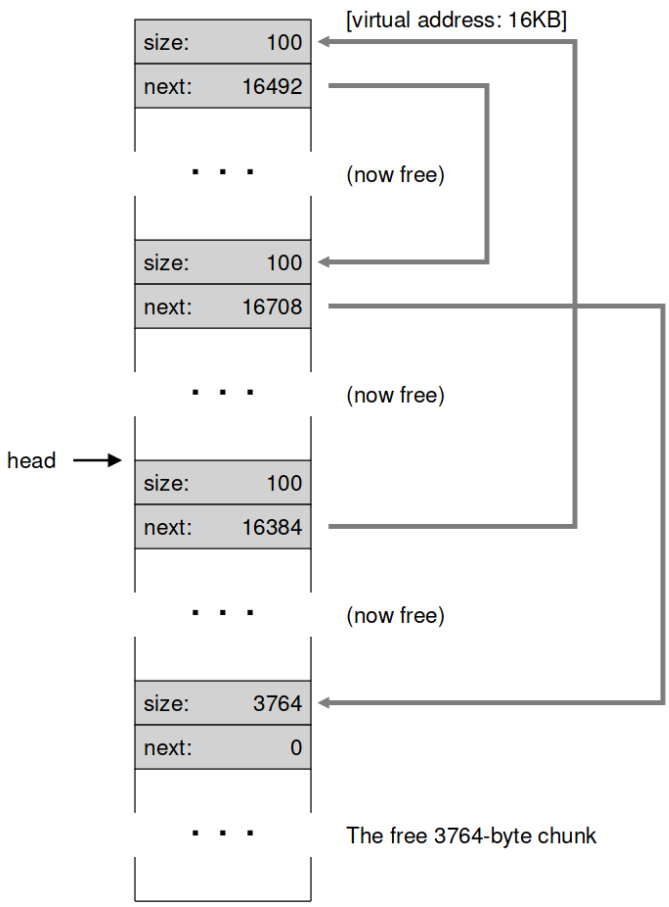


Figure 17.7: **A Non-Coalesced Free List**

Теперь вся память в нашей куче свободна, но вот проблема - свободные регионы не объединены в один и таким образом список свободных регионов побит на 4 части. Решение проблемы довольно просто - нужно пройтись по списку и объединить эти участки в один.

Growing the heap (Увеличение размера кучи)

Следует обсудить еще один механизм который используется во многих библиотеках выделения памяти. Проблема звучит так - что делать, когда в куче кончается место? Наиболее простой ответ - вернуть ошибку. В частности, ответив NULL.

Большинство традиционных библиотек выделения памяти начинают с кучи небольшого размера, а затем запрашивают у ОС чуть больше памяти, когда выделенная ранее память кончается. В типичном случае это означает, что библиотека выполняет некоторый системный вызов для увеличения кучи, а затем выделения в этой увеличенной куче новых кусочков памяти. Таким вызовом может быть например sbrk в Unix-подобных ОС. Для выполнения вызова sbrk ОС находит свободные физические страницы и отображает их в адресное пространство запросившего процесса, а затем возвращает значение конца новой кучи. В этот момент становится доступна куча большего размера и новые запросы могут быть успешно обслужены.

Базовые стратегии

Теперь, когда мы имеем некоторое понятие о механизме, рассмотрим некоторые базовые стратегии управления свободным пространством.
Идеальный распределитель должен быть одновременно и быстрым и минимизировать фрагментацию. К сожалению из-за того что и выделение памяти и ее освобождение может быть беспорядочным, любая конкретная стратегия может показывать плохие результаты в зависимости от входных данных. Таким образом мы не будем описывать идеальный подход, но поговорим о некоторых основах и обсудим их хорошие и плохие стороны.

Best fit

Стратегия наилучшего подходящего (best fit) довольно проста - на первом этапе пройтись через free list и найти свободные большие куски памяти или которые больше чем запрашиваемый размер региона выделяемой памяти. Затем в итоге выделить тот, который был наименьшего размера из этой группы кандидатов. Выделенный кусок называют еще best-fit chunk (наиболее-подходящий кусок). Одного прохода по списку свободных регионов достаточно чтобы найти такой кусок.

Однако, проблема в цене такого подхода: поиск подходящего курска является достаточно трудоемким процессом.

Worst fit

Подход, называемый worst fit является противоположным best fit, и заключается в следующем: найти наибольший свободный кусок в списке свободных и затем выделить из него запрашиваемый регион памяти. Оставшееся свободное место будет уменьшено на выделенный регион.

В таком подходе однако по-прежнему сохранена необходимость проходиться по всему списку свободных регионов и поэтому такой подход также плох в производительности. Даже хуже - исследования показывают, что такой подход может приводить к чрезмерной фрагментации и накладным расходам.

First Fit

Метод первого найденного самый простой. Он просто находит первый блок, который подходит по размеру запрошенного региона. Оставшееся свободное место будет уменьшено на выделенный регион.

Одним из хороших моментов такого подхода - скорость - нет накладных расходов на поиск нужного куска во всем списке. Однако, иногда может нагадить кучей маленьких выделенных объектов. Таким образом то, в какой именно очередности выделяется место в куче становится проблемой. Одним из подходов к решению - использовать порядок

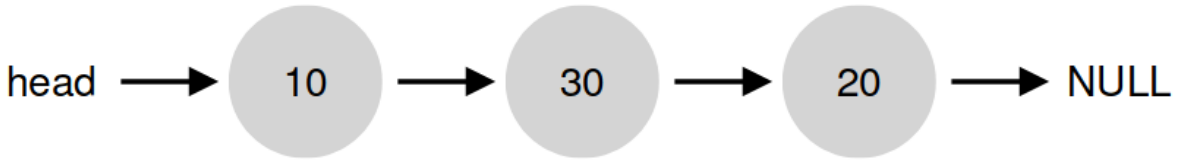
сортировки элементов во free-list согласно адресу. При этом в том числе объединение свободных участков становится проще, а фрагментация уменьшается.

Next Fit

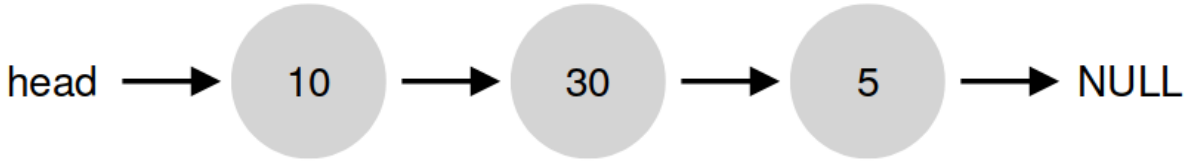
Следующий вариант похож на предыдущий, но вместо того чтобы отдавать первый найденный регион памяти достаточного размера, алгоритм хранит дополнительный указатель на то место, где он в последний раз искал. Идея заключается в том, чтобы распределять поиски подходящего региона по списку, таким образом избегая скопления в начале списка. Скорость работы похожа также на предыдущий алгоритм, поскольку опять же нет необходимости проходиться целиком по списку.

Примеры

Вообразим себе free list с тремя элементами в нем, размера 10,30, 20.



Предположим что программа запросила регион размером 15. Best-fit в данном случае пройдет через весь список и подберет регион размера 20, поскольку он подходит лучше всего. Результат:



Как результат - в данном случае получается маленький свободный регион размером 5. Подход worst fit подберет регион размера 30, и получится такой результат:



First fit в данном случае будет себя вести также как и worst fit. Вопрос будет только в цене поиска, first fit будет дешевле, т.к. Остановится сразу после того как найдет подходящий кусок.

Другие подходы

Кроме описанных базовых подходов существует еще множество других техник и алгоритмов, призванных улучшить и ускорить выделение памяти.

Обособленные списки

Одним из интересных подходов является создание нескольких обособленных списков. Если у какого-либо приложения есть множество запросов на выделение и освобождение регионов памяти одинакового размера, то на каждый такой размер создается один список, который и обслуживает такие запросы. Если пришел запрос на регион другого размера, то он будет передан основному списку.

Плюсы такого подходна очевидны - при условии выделения регионов одного размера фрагментация в списке гораздо меньше и таким образом запросы на освобождение памяти могут выполняться гораздо быстрее, а запросы на выделение региона нужного размера происходят гораздо быстрее, поскольку нет необходимости искать свободный кусок.

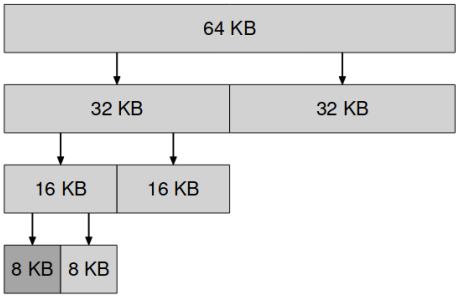
Однако как и в любой хорошей идее такой подход несет за собой и несколько проблем. Например, какое количество памяти следует отложить из общего региона памяти под запросы кусков одинакового размера, а сколько оставить для основного списка. Один из конкретных распределителей, называемый **slab allocator**, написанный инженером Jeff Bonwick был разработан для использования в ядре ОС solaris имеет следующий подход к решению.

Когда загружается ядро, оно выделяет некоторое количество кешей для объектов ядра, которые будут часто запрашиваться, например файлов блокировок или inode файловой системы. Каждый такой кеш является обособленным free list фиксированного размера. Они обслуживают запросы на выделение и освобождение достаточно быстро. Когда в таком кеше заканчивается место, выполняется запрос на выделение большего объема памяти под данный регион. И наоборот, когда количество ссылок на данный регион сводится к нулю, то основной распределитель может забрать этот регион под другие нужды. Такое может происходить, например, когда основной системе может понадобиться больше памяти.

Buddy Allocation (Распределение друзей)

Поскольку объедение является критичным для распределителя, один из подходов был предложен в так называемом **binary buddy allocator**.

В такой системе регион памяти выделенный под кучу считается некторой степенью числа 2. Когда выполняется запрос на выделение памяти, свободное пространство рекурсивно делится пополам до нахождения минимально подходящего куска. Такой найденный кусок и выдается запросившей програме. Пример разделения 64Кб для выделения 7Кб-ного блока.



В итоге пользователю будет выделен кусок размером 8Кб. Следует заметить что такой подход - явный пример внутренней фрагментации, поскольку остается неиспользуемое место.

Прелесть такого подхода проявляется когда кусок памяти освободили. При возвращении блока 8кб будет проверено, нет ли рядом другого свободного блока размером 8Кб, и если есть то они объединятся в 16кб. Затем будет выполнена проверка на наличие блока 16кб. Такое рекурсивное объединение продолжается до тех пор пока не найдется подходящего соседа или не будет освобожден весь лист. Скорость заключается в том что у соседних блоков адреса будут отличаться всего на один бит, который определяется уровнем дерева в котором расположен данный кусок памяти.